

SA Time Stability Tools

SA Long-Time-Measurement Stability & Phase-Noise Analysis Tool

Python 3.8 / 3.12-compatible toolkit for computing NIST SP1065 / SP559 time- and frequency-domain frequency-stability statistics, deriving SSB phase noise, classifying power-law noise types, converting vendor spectrum-analyzer exports, and producing Stable32- and Veusz-compatible outputs — wrapped in a PySide6 desktop GUI.

Contents

File	Purpose
<code>stability_core.py</code>	Canonical CSV I/O, drift fitting, ADEV/OADEV/MDEV/TDEV/OHDEV/Theo1/ThéoBR/ThéoH estimators, time-domain noise classification.
<code>phase_noise.py</code>	SSB phase noise $L(f)$ via Welch PSD, direct-trace parsing, frequency-domain noise classification, $L(f) \leftrightarrow Sy(f)$ conversion.
<code>sa_convert.py</code>	Keysight / Rohde & Schwarz / generic spectrum-analyzer export → canonical CSV converter, plus batch marker/peak time-series extraction.
<code>stable32_export.py</code>	Stable32-compatible ASCII <code>.PHD</code> / <code>.FRD</code> / <code>.DAT</code> file writer.
<code>veusz_export.py</code>	Veusz <code>.vszh5</code> multi-page plot export (run as an isolated subprocess — see caveat below).
<code>gui_app.py</code>	PySide6 desktop GUI tying all modules together.
<code>validate_*.py</code>	Development validation/smoke-test scripts (see "Validation performed" below).
<code>requirements.txt</code>	Pinned/version-gated dependency list, dual 3.8/3.12 compatible.

Installation

```
python3.12 -m pip install --break-system-packages -r requirements.txt
```

Veusz requires system Qt6 build tools that are **not** pulled in by pip automatically (see `requirements.txt` comments for the exact `apt-get` / `pip` sequence used and validated during development). If you only need the ADEV/phase-noise/Stable32/CSV-conversion functionality and

not the Veusz plot export, you may skip installing `veusz` and PyQt6 entirely — every other module has no dependency on it.

Running the GUI

```
python3.12 gui_app.py
```

Workflow: **Load Canonical CSV** (or **Convert Vendor SA File**) → set `tau0` (auto-filled from the loaded timestamps) and `f0` (nominal carrier/reference frequency, needed for phase-noise scaling) → **Remove Drift** (optional, order 0/1/2) → **Run Stability Analysis** and/or **Run SSB Phase Noise** → export via **Export Stable32 File** / **Export Veusz** (.vszh5).

Canonical CSV format

Two-column CSV with a header naming the value column:

```
time_s,phase_s      # phase / time-error data (seconds), OR
time_s,freq_hz      # absolute frequency readings (Hz), OR
time_s,frac_freq    # fractional frequency y = (f - f0)/f0 (dimensionless)
```

`time_s` must be monotonically increasing; near-uniform spacing is assumed for the ADEV-family estimators (`tau0` is taken as the median sample spacing). `stability_core.load_canonical_csv()` auto-detects which value column is present and returns its type so callers can convert to phase as needed (`phase_to_freq()` / `freq_to_phase()`).

Vendor spectrum-analyzer conversion — important caveat

Exact SA export layouts vary by instrument model and firmware revision. `sa_convert.py` implements:

- a **generic fallback parser** that auto-detects delimiter (, / ; /tab), skips metadata/comment lines, and classifies column 0 as a time series (near-uniform spacing) or a frequency sweep/trace (log-uniform spacing, large dynamic range, no zero values) — validated against synthetic files mimicking both cases (`validate_sa_convert.py`);
- **Keysight-** and **R&S-**flavored parsers that locate the first contiguous numeric data block after a free-text/metadata header, since

neither vendor uses one single fixed header format across firmware versions and product lines (PXA/MXA/N-series vs FSW/FSVA/FSWP).

Before trusting a bulk conversion on real instrument data, export one small sample file from your specific instrument/firmware and confirm the value units (Hz vs dBc/Hz vs dBm, phase vs frequency) match what the converter classified it as. Extend `_VENDOR_PARSERS` in `sa_convert.py` with a new function if your file layout doesn't match either built-in vendor parser — the module docstring documents the extension point.

`batch_extract_marker_time_series()` builds a long-time-measurement series from repeated single-sweep trace snapshots (peak/marker tracking) when your SA workflow captures individual snapshots rather than a native zero-span/marker-log time series; it assumes uniform capture spacing unless you re-map the returned index array to real elapsed time from your own capture timestamps.

Stable32 export format

Implements the ASCII data-file convention documented in the Stable32 manual [12]: plain text, tab-delimited `timetag value` pairs, double-precision exponential notation, and the "gap = 0" convention (a missing point is written as exactly `0`, and Stable32 forbids a gap at the first or last data point — `write_stable32_phase_file()` / `write_stable32_freq_file()` raise `ValueError` if you attempt this rather than silently producing an unreadable file). A `#`-prefixed header block (File/Date/Type/Title/Columns) is written for documentation; verify your specific Stable32 version tolerates comment lines if strict compliance matters, since the base format does not formally define them.

Veusz export — process isolation note

`veusz.embed` loads **PyQt6** internally. This GUI uses **PySide6**. Loading both Qt bindings in the *same* process risks a Qt runtime conflict (duplicate `QApplication`, plugin/symbol clashes). `gui_app.py` therefore never imports `veusz_export` directly — it always calls `veusz_export.export_all_plots_subprocess()`, which serializes the plot data to a temp JSON file and runs the export in a fresh `python3.12 veusz_export.py --payload ... subprocess`. If you use `veusz_export.py` outside the GUI, prefer `export_all_plots_subprocess()` over `export_all_plots()` for the same reason unless you are certain PySide6 has not been imported in your process.

Headless rendering requires `QT_QPA_PLATFORM=offscreen` ; both `veusz_export.py` and `gui_app.py` set this automatically if not already set in the environment.

Time-domain noise classification — reference table

Noise type	ADEV slope μ	MDEV slope μ
White PM	−1.0	−1.5
Flicker PM	−1.0	−1.0
White FM	−0.5	−0.5
Flicker FM	0.0	0.0
Random Walk FM	+0.5	+0.5
Flicker Walk FM	+1.0	+1.0
Linear Frequency Drift	+1.5	+1.5

(`stability_core.NOISE_TABLE` , per [1] Sec. 3.4, [2].)

Frequency-domain (SSB phase noise) classification — reference table

Noise type	L(f) slope (dB/decade)
White PM	0
Flicker PM	−10
White FM	−20
Flicker FM	−30
Random Walk FM	−40

(`phase_noise.FREQ_NOISE_TABLE` , per [1] Table 1 / [11] Ch. 2.) Because $L(f) = 10 \cdot \log_{10}(0.5 \cdot S_{\varphi}(f))$ is already expressed in dB, its slope against $\log_{10}(f)$ is **10× the underlying $S_{\varphi}(f)$ power-law exponent** — this factor-of-10 scaling was confirmed numerically against synthetic white-FM/flicker-FM/random-walk-FM series in `validate_phase_noise.py` (measured slopes: −19.2, −29.2, −39.2 dB/decade vs. the −20/−30/−40 textbook values, within expected Welch-estimator variance) after an initial implementation bug compared L(f) slope directly against the un-scaled exponent table and is documented in the module for future maintainers. Because a single Welch periodogram has high per-bin variance, `classify_freq_domain_noise()` **log-bin-averages** L(f) before

computing local slopes (`_log_bin_average()`), matching standard commercial phase-noise-analyzer practice; even so, treat single adjacent-bin classifications as a noisy per-bin trend, not a definitive per-bin verdict — look at the overall trend across a full decade.

Equation appendix (LaTeX)

Overlapping Allan deviation (OADEV), phase form [1], [2]:

$$\sigma_y^2(m\tau_0) = \frac{1}{2(m\tau_0)^2(N-2m)} \sum_{i=1}^{N-2m} (x_{i+2m} - 2x_{i+m} + x_i)^2$$

Modified Allan deviation (MDEV), phase form [1]:

$$\text{Mod } \sigma_y^2(m\tau_0) = \frac{1}{2(m\tau_0)^2 m^2 (N-3m+1)} \sum_{j=1}^{N-3m+1} \left[\sum_{i=j}^{j+m-1} (x_{i+2m} - 2x_{i+m} + x_i) \right]^2$$

Time deviation (TDEV) [1]:

$$\sigma_x(\tau) = \frac{\tau}{\sqrt{3}} \text{Mod } \sigma_y(\tau)$$

Overlapping Hadamard deviation (OHDEV), phase form [1]:

$$\sigma_H^2(m\tau_0) = \frac{1}{6(m\tau_0)^2(N-3m)} \sum_{i=1}^{N-3m} (x_{i+3m} - 3x_{i+2m} + 3x_{i+m} - x_i)^2$$

Theo1 (corrected form, [7], [8]), for even (m), ($10 \leq m \leq N-1$):

$$\text{Theo1}(m, \tau_0, N) = \frac{1}{0.75(N-m)(m\tau_0)^2} \sum_{i=0}^{N-m-1} \sum_{w=1}^{m/2} \frac{[(x_i - x_{i+w}) + (x_{i+m} - x_{i+m-w})]^2}{w}$$

with effective averaging time

$$\tau_{\text{eff}} = 0.75 m \tau_0 \quad (\text{not } m\tau_0).$$

ThêoBR / ThêoH bias correction (Howe ladder-ratio method, [6], [13]):

$$G = \left\langle \frac{\sigma_y^2(m_{\text{AVAR}})}{\text{Theo1}(m_{\text{Theo}})} \right\rangle_{\text{ladder pairs}}, \quad \text{ThêoBR}(m) = \sqrt{G} \cdot \text{Theo1}(m)$$

evaluated at ($\tau = 0.75, m, \tau_0$); ThêoH splices AVAR (for ($m < k = 0.1N$)) with ThêoBR (for ($m \geq k/0.75$)).

SSB phase noise [1], [11]:

$$\phi(t) = 2\pi f_0 x(t), \quad L(f) = 10 \log_{10} \left[\frac{1}{2} S_\phi(f) \right] \text{ dBc/Hz}$$

where $(S_\phi(f))$ is the (Welch [10]) power spectral density of $\phi(t)$.

$L(f) \leftrightarrow$ fractional-frequency PSD [1]:

$$S_\phi(f) = 2 \cdot 10^{L(f)/10},$$

$$S_y(f) = \frac{f^2}{f_0^2} S_\phi(f)$$

Linear/quadratic drift removal (least-squares polynomial fit to $(x(t))$, residual used for all deviation estimators above):

$$x(t) = a_0 + a_1 t + a_2 t^2 + \varepsilon(t)$$

Validation performed

All numeric claims below were verified in this development session, not merely asserted:

- `validate_core.py`: OADEV/MDEV/OHDEV/TDEV matched the independent `allantools` [9] library to 0.0000% error on synthetic data; Theo1 confirmed unbiased vs. OADEV at the correct effective tau (ratio 0.97–1.00) for white FM noise per [7], [8]; ThéoH hybrid splice extended usable tau coverage to 62.8% of record length vs. 50% for OADEV alone; 5-of-5 canonical noise types correctly classified from ADEV/MDEV slopes.
- `validate_phase_noise.py`: $L(f)$ slopes for synthetic White/Flicker/Random-Walk FM phase series measured at $-19.2/-29.2/-39.2$ dB/decade vs. the $-20/-30/-40$ textbook targets (within Welch-estimator variance); `l_f_to_sy()` round-trip produced finite, positive $S_y(f)$ values.
- `validate_sa_convert.py`: generic/Keysight/R&S parsers correctly distinguished synthetic time-series vs. frequency-trace inputs in all 4 test files after fixing an initial classifier bug that misfired on time series starting at $t=0$; batch marker extraction recovered the correct peak values from 6 synthetic snapshot files.
- Stable32 export: round-tripped a synthetic phase series through `write_stable32_phase_file()` / `write_stable32_freq_file()` / `write_stable32_sigma_tau_table()` without error; first/last-point gap prohibition confirmed to raise `ValueError` as designed.
- `validate_veusz_export.py`: full pipeline (real `stability_core` + `phase_noise` outputs $\rightarrow$ `veusz_export.export_all_plots_subprocess()`) produced a valid non-trivial `.vszh5` file (87–174 KB across test

runs) with no exceptions, run via the isolated-subprocess path used by the GUI.

- `validate_gui.py`: `headless ( QT_QPA_PLATFORM=offscreen )` smoke test drove `MainWindow`'s CSV load, drift removal, stability analysis, phase-noise analysis, and both export paths end-to-end without error, populating both result tables with real computed rows.

Known limitation: No Python 3.8 interpreter was available in this development sandbox to directly execute the dual-version test suite;

3.8 compatibility is enforced only through syntax discipline (`from __future__ import annotations`, `typing.List/Optional/Tuple` instead of PEP 604/585 syntax, no `match / case`, no `dataclass kwonly`) and version-gated `requirements.txt` markers, not by an actual 3.8 test run. Before deploying to a Python 3.8 environment, run the `validate*.py`

scripts there directly to confirm.

IEEE reference table

[1] W. J. Riley, "Handbook of Frequency Stability Analysis," NIST Special Publication 1065, Jul. 2008. Available: <https://tf.nist.gov/general/pdf/2220.pdf>

[2] J. E. Kamas and S. R. Stein, Eds. (orig. D. A. Howe, D. W. Allan, J. A. Barnes), "Characterization of Clocks and Oscillators," NIST Special Publication 559/updated as SP1065 predecessor; see also D. W. Allan, "Time and Frequency (Time-Domain) Characterization, Estimation, and Prediction of Precision Clocks and Oscillators," NIST, 1990. Available: <https://tf.nist.gov/general/pdf/461.pdf>

[3] "IEEE Standard Definitions of Physical Quantities for Fundamental Frequency and Time Metrology — Random Instabilities," IEEE Std 1139-2008, doi: 10.1109/IEEESTD.2008.4797525.

[4] D. A. Howe, "ThêoH: A Hybrid, High-Confidence Statistic that Improves on the Allan Deviation," *Metrologia*, vol. 43, no. 4, pp. S322–S327, 2006, doi: 10.1088/0026-1394/43/4/S17.

[5] D. A. Howe and F. Vernotte (T. N. Tasset), "Thêo1: Characterization of Very Long-Term Frequency Stability," in Proc. 18th Eur. Freq. Time Forum (EFTF), 2004; NIST report. Available: <https://tf.nist.gov/general/pdf/1894.pdf> and <https://tf.boulder.nist.gov/general/pdf/1979.pdf>

[6] Purdue University OxideMEMS Lab, "ThêoH Bias Removal," in Proc. 2006 IEEE Int. Freq. Control Symp. (FCS). Available: https://engineering.purdue.edu/oxidemems/conferences/fcs2006/PDFs/Papers/145_6180.pdf

[7] D. A. Howe and T. N. Tasset (as [5]) — corrected Theo1 closed form used in `stability_core.theo1()`.

[8] IEEE Std 1139-2008 (as [3]) and NIST SP1065 [1] Eq. 30 — Theo1 effective-tau relation ($\tau_{\mathrm{eff}} = 0.75 \tau_0$).

[9] "Allantools Documentation." Available:

<https://allantools.readthedocs.io/en/latest/functions.html>

[10] P. Welch, "The Use of Fast Fourier Transform for the Estimation of Power Spectra," IEEE Trans. Audio Electroacoust., vol. 15, no. 2, pp. 70–73, Jun. 1967, doi: 10.1109/TAU.1967.1161901.

[11] E. Rubiola, "Phase Noise and Frequency Stability in Oscillators," Cambridge Univ. Press, 2008, doi: 10.1017/CBO9780511812798.

[12] "Stable32 Software Manual," Hamilton Technical Services, rev. 1.54.

Available: <http://www.stable32.com/Manual154.pdf>; see also

W. J. Riley, "Frequency Stability Analysis Using Stable32," application note. Available:

<http://www.wriley.com/Learn%20Frequency%20Stability%20Analysis%20Using%20Stable32.pdf>

[13] D. A. Howe, "ThêoH ladder-ratio bias-factor method" (as [4], [6]).

[14] J. A. Barnes, A. R. Chi, L. S. Cutler, D. J. Healey, D. B. Leeson, T. E. McGunigal, J. A. Mullen, W. L. Smith, R. L. Sydnor, R. F. C. Vessot, and G. M. R. Winkler, "Characterization of Frequency Stability," NBS Technical Note 394 / IEEE Trans. Instrum. Meas., vol. IM-20, no. 2, pp. 105–120, May 1971.

[15] Keysight Technologies, "PXA/MXA Signal Analyzer Phase Noise Measurement Application User's Guide," Keysight Technologies.

[16] Rohde & Schwarz, "R&S FSWP Phase Noise Analyzer and VCO Tester User Manual," Rohde & Schwarz GmbH & Co. KG.

[17] "PySide6 Documentation," Qt for Python. Available:

<https://doc.qt.io/qtforpython-6/>

[18] "Veusz Documentation," Veusz project. Available:

<https://veusz.github.io/docs/>

[19] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," Comput. Sci. Eng., vol. 9, no. 3, pp. 90–95, 2007, doi: 10.1109/MCSE.2007.55.

[20] Microsemi/Microchip, "MHM-2010 / MSM-2010 Hydrogen Maser User Guide," Rev. J, Microchip Technology Inc.